

Backend Test # 10

You've survived the test. The results will arrive soon. 🎉

Your score: 91% 24.5 / 27

Duration: 0:17:00

1.

What is the output?

1 / 1 point

```
List<Integer> list = Arrays.asList(10, 20, 30, 40);

long count = list.stream().filter(x -> x > 20).count();

System.out.println(count);
```

Your Answer: ✓ Correct

☒ 2

☐ 3

☐ 4

☐ Compilation error

Explanation:

Practice today. Confidence tomorrow. Job soon!

2.

Which Stream API method is used to transform each element?

1 / 1 point

Your Answer: ✓ Correct

☐ filter()

☒ map()

☐ reduce()

☐ collect()

3.

What does distinct() do in a Stream?

1 / 1 point

```
List<Integer> list = Arrays.asList(10, 20, 10, 30, 40);

System.out.println(list.stream().distinct().collect(Collectors.toList()));
```

Your Answer: ✓ Correct

☐ Sorts the elements

☒ Removes duplicate elements

☐ Removes null elements

☐ Converts elements to String

4.

Which of the following is a terminal operation?

1 / 1 point

Your Answer: ✓ Correct

☐ filter()

☐ map()

☐ sorted()

☒ collect()

5.

What is the main purpose of flatMap()?

1 / 1 point

Your Answer: ✓ Correct

☐ Sort elements

☐ Remove duplicates

☒ Flatten nested structures into a single Stream

☐ Count elements

6.

List<Integer> list = Arrays.asList(10, 20, 30);

1 / 1 point

```
Stream<Integer> stream = list.stream().filter(x -> x > 10);

System.out.println(stream.count());

System.out.println(stream.count());
```

output?

Your Answer: ✓ Correct

☐ Prints 2 and 2

☐ Prints 3 and 3

☐ Compilation error

☒ Runtime Exception

7.

Which statement about Java Streams is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ A Stream stores the elements of a collection internally.

☐ A Stream modifies the original collection by default.

☒ A Stream represents a pipeline of operations for processing data.

☐ A Stream can be reused after a terminal operation.

8.

Which statement about intermediate operations is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ They execute immediately when called.

☐ They always return a Collection.

☒ They are lazy and generally execute only when a terminal operation is invoked.

☐ They can be called only once in a Stream pipeline.

9.

Which statement correctly describes map() and flatMap()?

1 / 1 point

Your Answer: ✓ Correct

☐ map() is used for sorting, while flatMap() is used for filtering.

☐ Both always produce exactly the same type of result.

☐ flatMap() can only be used with primitive Streams.

☒ map() transforms each element into another element, while flatMap() transforms and flattens multiple resulting Streams.

10.

Which statement about Stream reuse is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ A Stream can be reused unlimited times if it has no terminal operation.

☐ A Stream can be reused after collect().

☒ Once a terminal operation has been invoked, the Stream is consumed and generally cannot be reused.

☐ A Stream can be reused only if it was created from an ArrayList.

11.

Which statement about parallel streams is TRUE?

1 / 1 point

Your Answer: ✓ Correct

☐ parallelStream() guarantees that elements will be processed in order.

☐ Parallel streams always improve performance compared to sequential streams.

☐ Parallel streams create one new thread for every element.

☒ Parallel streams can process elements concurrently, but the actual performance benefit depends on the workload and other factors.

12.

A Java application has 5 threads running on a system with 1 CPU core. All 5 threads are in the RUNNABLE state.

1 / 1 point

What is primarily responsible for deciding which thread gets CPU time next?

Your Answer: ✓ Correct

☐ JVM Garbage Collector

☒ Java Thread Scheduler

☐ Java Compiler

☐ JDK

13.

A program creates two threads:

1 / 1 point

```
Thread t1 = new Thread();

Thread t2 = new Thread();

t1.start();

t2.start();
```

The developer expects t1 to always finish before t2 starts because t1.start() is called first.

Which statement is correct?

Your Answer: ✓ Correct

☐ t1 will always execute completely before t2.

☐ t2 will never execute until t1 finishes.

☒ The scheduler decides when each runnable thread gets CPU time; calling start() first does not guarantee execution order.

☐ Both threads execute simultaneously on every system.

14.

A computer has only 1 CPU core and a Java application has 3 runnable threads. Which statement is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ All 3 threads execute CPU instructions at exactly the same time.

☒ Only one thread can execute on the CPU at a given instant, while the scheduler switches CPU time among the threads.

☐ Java automatically creates 3 CPU cores for the 3 threads.

☐ Threads cannot be used on a single-core CPU.

15.

final List<Integer> list = new ArrayList<>();

-0.25 / 1 point

```
list.add(10);

list.add(20);
```

What happens here?

Your Answer: ✗ Incorrect

☒ Compilation error at list.add(10)

☐ Compilation error because final objects cannot be modified

☒ Successfully adds the elements

☐ Runtime exception

16.

What happens when the following code executes?

1 / 1 point

```
class Parent {

    Parent() {

        System.out.println("Parent");

    }

}

class Child extends Parent {

    Child() {

        System.out.println("Child");

    }

}

Child c = new Child();
```

Your Answer: ✓ Correct

☐ Child

☐ Parent

☒ Parent followed by Child

☐ Compilation error

17.

What is the primary reason a HashSet does not allow duplicate elements?

1 / 1 point

Your Answer: ✓ Correct

☐ It uses == to compare elements.

☒ It uses hashCode() and equals() to determine equality.

☐ It automatically sorts elements.

☐ It stores elements using indexes.

18.

class Test {

1 / 1 point

```
    static void show(int x) {

        System.out.println("int");

    }

    static void show(long x) {

        System.out.println("long");

    }

    public static void main(String[] args) {

        short s = 10;

        show(s);

    }

}
```

What will be the output?

Your Answer: ✓ Correct

☐ short

☒ int

☐ long

☐ Compilation error

19.

A Student class needs to support sorting by name, by age, and by marks, depending on the requirement. Which approach is most appropriate?

1 / 1 point

Your Answer: ✓ Correct

☐ Implement Comparable<Student> three times in the Student class.

☒ Use multiple Comparator<Student> implementations/lambda for the different sorting criteria.

☐ Use only compareTo() because Java allows multiple compareTo() methods.

☐ Override equals() for each sorting requirement.

20.

Which statement about a Functional Interface is correct?

1 / 1 point

Your Answer: ✓ Correct

☐ It must contain exactly one method, including methods inherited from Object.

☒ It must contain exactly one abstract method, but it can contain multiple default and static methods.

☐ It cannot contain default methods.

☐ It cannot be used with lambda expressions.

21.

Which of the following is a marker interface in Java?

1 / 1 point

Your Answer: ✓ Correct

☐ Runnable

☐ Comparable

☒ Serializable

☐ Callable

22.

Which statement is correct about ConcurrentHashMap in Java?

1 / 1 point

Your Answer: ✓ Correct

☐ It allows one null key and multiple null values, just like HashMap.

☐ Its get() operation locks the entire map to ensure thread safety.

☒ It provides thread-safe concurrent access without locking the entire map for ordinary operations, and it does not allow null keys or null values.

☐ It guarantees that iteration will always reflect a completely consistent snapshot of the map.

23.

import java.util.*;

-0.25 / 1 point

```
public class Test {

    public static void main(String[] args) {

        Set<String> set = new TreeSet<>((a, b) -> a.length() - b.length());

        set.add("Java");

        set.add("Spring");

        set.add("Boot");

        set.add("API");

        System.out.println(set);

    }

}
```

Your Answer: ✗ Incorrect

☒ [API, Boot, Java, Spring]

☐ [API, Java, Spring]

☐ [Java, Boot, API, Spring]

☐ [Spring, Java, Boot, API]

24.

Which of the following statements about equals() and hashCode() is CORRECT?

1 / 1 point

Your Answer: ✓ Correct

☐ If two objects have the same hashCode(), their equals() method must return true.

☒ If equals() returns true for two objects, their hashCode() values must be the same.

☐ If two objects have different hashCode() values, their equals() method may still return true.

☐ Overriding equals() without overriding hashCode() has no impact when objects are used as keys in HashMap.

25.

class Student

1 / 1 point

```
{

    int id;

    String name;

    int marks;

}
```

The application requires:

- By default, students should be sorted by id.
- On one particular screen, they should be sorted by marks in descending order.
- On another screen, they should be sorted by name alphabetically.

Which design is most appropriate?

Your Answer: ✓ Correct

☐ Implement Comparable<Student> and create separate compareTo() methods for id, marks, and name.

☐ Use only Comparator<Student> because Comparable supports only primitive fields.

☐ Implement Comparable<Student> for the default id ordering and use separate Comparator<Student> implementations/lambda for marks and name.

☒ Implement Comparable<Student> for the default id ordering and use separate Comparator<Student> implementations/lambda for marks and name.

26.

Consider the following code:

1 / 1 point

```
import java.util.*;

class Employee {

    int id;

    Employee(int id) {

        this.id = id;

    }

    @Override

    public int hashCode() {

        return id % 5;

    }

    @Override

    public boolean equals(Object obj) {

        Employee e = (Employee) obj;

        return this.id == e.id;

    }

}

public class Test {

    public static void main(String[] args) {

        Map<Employee, String> map = new HashMap<>();

        Employee e1 = new Employee(10);

        Employee e2 = new Employee(15);

        Employee e3 = new Employee(10);

        map.put(e1, "A");

        map.put(e2, "B");

        map.put(e3, "C");

        System.out.println(map.size());

        System.out.println(map.get(new Employee(10)));

    }

}
```

What will be the output?

Your Answer: ✓ Correct

☐ 2

☐ A

☒ 2

☐ C

☐ 3

☐ C

☐ 3

☐ null

27.

Consider this design:

1 / 1 point

```
class PaymentService {

    public void pay(String type, double amount) {

        if (type.equals("CREDIT_CARD")) {

            // Credit card payment logic

        }

        else if (type.equals("UPI")) {

            // UPI payment logic

        }

        else if (type.equals("NET_BANKING")) {

            // Net banking payment logic

        }

    }

}
```

Tomorrow, the application needs to support PayPal. The developer must modify the existing PaymentService and add another else-if block.

Which SOLID principle is primarily violated?

Your Answer: ✓ Correct

☐ Single Responsibility Principle (SRP)

☒ Open/Closed Principle (OCP)

☐ Liskov Substitution Principle (LSP)

☐ Interface Segregation Principle (ISP)